TRAVAUX DIRIGÉS — INFORMATIQUE

Géolocalisation, calcul de distances et triangulation en Python

Ce TD s'inscrit dans une problématique de géolocalisation appliquée : déterminer la position d'un point inconnu à partir de distances mesurées depuis plusieurs points de référence (triangulation). Vous travaillerez avec des données réelles de villes françaises et des scripts Python utilisant les bibliothèques folium et pandas.

Objectifs pédagogiques

- Lire et exploiter un fichier CSV avec Python (modules csv et pandas)
- Générer des cartes interactives avec la bibliothèque folium
- Appliquer le principe de triangulation géographique
- Analyser, corriger et améliorer du code Python existant

Fichiers fournis

Fichier	Description
`villes_coo.csv`	Base de données de villes françaises avec coordonnées GPS
`villes_vol_oiseau01.py`	Script v1 : calcul de distance et tracé du segment sur la carte
`villes_vol_oiseau02.py`	Script v2 : ajout du marqueur de distance sur la carte
`triangulation.py`	Script de triangulation à 3 villes avec saisie utilisateur
`triangulation_autre_approche.py`	Refactorisation du script de triangulation en fonctions

PARTIE 1 — DISTANCE À VOL D'OISEAU

Rappels théoriques — Formule de Haversine

La formule de Haversine calcule la distance en km entre deux points GPS sur la sphère terrestre (R ≈ 6371 km). Soient P1(lat1, lon1) et P2(lat2, lon2) (coordonnées en radians) :

```
dlat = lat2 - lat1
dlon = lon2 - lon1
a = sin(dlat/2)² + cos(lat1) × cos(lat2) × sin(dlon/2)²
c = 2 × atan2(sqrt(a), sqrt(1-a))
distance = R × c
```

La conversion degrés → radians s'effectue avec math.radians() en Python.

Exercice 1.1 — Analyse des scripts fournis

Q1. Ouvrez le fichier **villes_coo.csv**. Quel est le séparateur utilisé ? Quelle est la structure des données (colonnes) ?

Q2. Comparez **villes_vol_oiseau01.py** et **villes_vol_oiseau02.py**. Quelle est l'unique différence fonctionnelle ? Pourquoi est-elle utile pour l'utilisateur de la carte ?

Q3. Dans **calculer_distance(),** pourquoi est-il nécessaire de convertir les coordonnées en radians ? Que se passerait-il sans cette conversion ?

Exercice 1.2 — Modification et extension du code

Q4. Réécrivez la fonction obtenir_coordonnees_de_csv() de **villes_vol_oiseau01.py** en utilisant le module csv standard (sans pandas). La fonction doit retourner None si la ville est introuvable.

Q5. Ajoutez une gestion d'erreur dans main() de **villes_vol_oiseau02.py** : si l'une des villes n'est pas trouvée, afficher un message précisant laquelle est manquante, sans faire planter le programme.

PARTIE 2 — TRIANGULATION GÉOGRAPHIQUE

Rappels théoriques — Principe de la triangulation

La triangulation localise un point inconnu X à partir de distances connues depuis 3 points de référence A, B, C. X se trouve à l'intersection des 3 cercles de centres A, B, C et de rayons d(X,A), d(X,B), d(X,C).

Deux cercles se coupent en 2 points (ambiguïté). Un 3e cercle lève cette ambiguïté. En pratique, les imprécisions font que les cercles délimitent une zone d'incertitude plutôt qu'un point exact.

Exercice 2.1 — Analyse du script triangulation.py

Q6. Quel est le rôle de **obtenir_coordonnees_de_csv()** dans **triangulation.py** ? En quoi diffère-t-elle de celle des scripts villes_vol_oiseau ?

Q7. Dans **triangulation.py**, le rayon saisi en km est multiplié par 1000. Expliquez pourquoi. Quelle est l'unité du paramètre radius dans folium ?

Q8. Pourquoi 3 cercles sont-ils nécessaires pour la triangulation ? Dans quels cas 2 cercles ne donnent-ils pas une solution unique ?

Exercice 2.2 — Amélioration du code de triangulation

Q9. Le fichier **triangulation_autre_approche.py** propose une refactorisation. Identifiez les 2 fonctions ajoutées et expliquez l'intérêt de cette décomposition.

Q10. **triangulation_autre_approche.py** contient un bug : dans add_circles_and_markers(), la variable coords est utilisée sans être passée en paramètre. Proposez une correction complète.

Q11. Réécrivez la boucle de saisie de **triangulation.py** avec une liste et une boucle for, en validant que les rayons soient strictement positifs.

PARTIE 3 — QUESTIONS D'ANALYSE DE CODE

Exercice 3.1 — Qualité et robustesse du code

Q12. Dans **triangulation.py**, que se passe-t-il si l'utilisateur entre "bayonne" au lieu de "Bayonne" ? Proposez une correction insensible à la casse.

Q13. Identifiez deux problèmes potentiels liés à l'encodage des fichiers dans les scripts fournis. Comment les scripts gèrent-ils (ou ne gèrent-ils pas) les caractères accentués ?

Q14. Que signifie **if __name__ == '__main__'**: dans les scripts ? Quel problème évite-t-elle lors de l'import d'un module ?

Exercice 3.2 — Question de synthèse

Q15. Proposez une architecture en modules Python pour un projet regroupant les fonctionnalités des 4 scripts. Décrivez les modules, leurs fonctions principales et leurs dépendances.

BONUS — Pour aller plus loin

Questions bonus (facultatives)

- Implémentez une recherche approximative dans le CSV avec difflib pour suggérer des noms proches si la ville n'est pas trouvée exactement.
- Ajoutez une interface CLI avec argparse pour passer les villes et rayons en arguments plutôt qu'en saisie interactive.
- Calculez et affichez automatiquement l'intersection approximative des 3 cercles en résolvant le système numériquement avec scipy.optimize.
- Exportez les résultats (villes, rayons, distances) dans un fichier JSON en plus de la carte HTML.